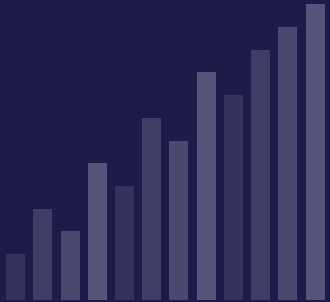
AULA 12 · BLOCO 2: ORDENAÇÃO AVANÇADA E ESTRUTURAS LINEARES

Quicksort aleatorizado e a análise do caso médio

Cormen et al., *Algoritmos*: seções 7.3 e 7.4



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Aleatorizar a escolha do pivô e entender o que isso garante
- Calcular o número esperado de comparações com variáveis indicadoras
- Distinguir caso médio (sobre entradas) de tempo esperado (sobre sorteios)
- Medir a concentração das comparações em torno da esperança

Teoria

Pivô sorteado, análise por pares comparados, $E[X] = O(n \lg n)$

Prática em C

Quicksort aleatorizado em C

Experimento

Entrada ordenada: pivô fixo contra pivô sorteado; histograma

Conexões

Algoritmos aleatorizados em redes, segurança, IA e sistemas distribuídos



Teoria

- Cormen et al., seções 7.3 e 7.4
- Aleatoriedade contra o adversário
- Quem é comparado com quem?
- Somando as probabilidades
- Partições boas são a regra

TEORIA

Aleatoriedade contra o adversário

- Nenhuma entrada específica provoca o pior caso: ele depende só do **sorteio**.
- O **tempo esperado** vale para **toda** entrada, inclusive a ordenada.
- Caso médio (aula 11) supunha entradas aleatórias; aqui a aleatoriedade está no algoritmo.

RANDOMIZED-PARTITION(A, p, r)

```
1 i ← RANDOM(p, r)
2 troque A[r] com A[i]
3 retorne PARTITION(A, p, r)
```

TEORIA

Quem é comparado com quem?

Renomeie os elementos em ordem: $z_1 < z_2 < \cdots < z_n$, e seja $Z_{ij} = \{z_i, \ldots, z_j\}$.

- Dois elementos são comparados **no máximo uma vez**: só com o pivô, que depois sai do jogo.
- z_i e z_j são comparados **se e somente se** o primeiro pivô escolhido em Z_{ij} é z_i ou z_j. Se for um elemento do meio, eles vão para lados diferentes e nunca se encontram.
- Todos os $j - i + 1$ elementos de Z_{ij} têm a mesma chance de ser o primeiro pivô:

$$\Pr\{z_i \text{ comparado com } z_j\} = \frac{2}{j - i + 1}$$

TEORIA

Somando as probabilidades

Com X_{ij} indicadora de "z_i é comparado com z_j" e $X = \sum X_{ij}$:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\lg n) = O(n \lg n)$$

Fazendo a conta exata: $E[X] = 2(n+1)H_n - 4n \approx 1{,}39\, n \lg n$.

> Linearidade da esperança: a esperança da soma é a soma das esperanças, mesmo que os X_{ij} sejam dependentes.

TEORIA

Partições boas são a regra

Com pivô aleatório, a chance de a partição ser pior que 1 para 9 é só 20%. Partições ruins ocasionais são absorvidas pelas boas: a recursão continua com profundidade $O(\lg n)$.

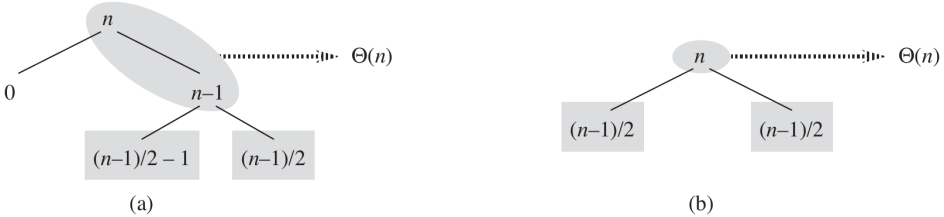


Figura 7.5 de Cormen et al. (2009) — (a) uma partição ruim seguida de uma boa custa o mesmo que (b) uma boa: o custo da ruim é absorvido.



Prática em C

- `rand()` sorteia o pivô em $[p, r]$
- `srand(time(NULL))`: sorteios diferentes a cada execução
- Entrada já ordenada deixa de ser problema

PRÁTICA EM C

Quicksort aleatorizado em C (1/3)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void troca(int A[], int i, int j) { int t = A[i]; A[i] = A[j]; A[j] = t; }

int particiona(int A[], int p, int r) {
    int x = A[r], i = p - 1;
    for (int j = p; j < r; j++)
        if (A[j] <= x) troca(A, ++i, j);
    troca(A, i + 1, r);
    return i + 1;
}
```

PRÁTICA EM C

Quicksort aleatorizado em C (2/3)

```c
/* RANDOMIZED-PARTITION: sorteia o pivô em A[p..r] e o leva para o fim. */
int particiona_aleatorio(int A[], int p, int r) {
    int k = p + rand() % (r - p + 1);
    troca(A, k, r);
    return particiona(A, p, r);
}

void quicksort_aleatorio(int A[], int p, int r) {
    if (p < r) {
        int q = particiona_aleatorio(A, p, r);
        quicksort_aleatorio(A, p, q - 1);
        quicksort_aleatorio(A, q + 1, r);
    }
}
```

PRÁTICA EM C

Quicksort aleatorizado em C (3/3)

```c

int main(void) {
    srand((unsigned)time(NULL));            // cada execução sorteia pivôs diferentes
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    int *A = malloc(n * sizeof(int));
    for (int k = 0; k < n; k++)
        if (scanf("%d", &A[k]) != 1) return 1;
    quicksort_aleatorio(A, 0, n - 1);
    for (int k = 0; k < n; k++) printf("%d%c", A[k], k < n - 1 ? ' ' : '\n');
    free(A);
    return 0;
}
```



Experimento

- Entrada ordenada: pivô fixo contra sorteado
- Pivô sorteado: média de 20 execuções por n
- Histograma de 20.000 execuções com $n = 1000$

EXPERIMENTO

O pior caso desaparece

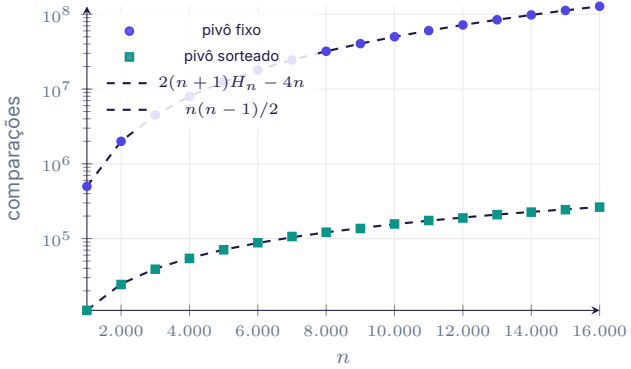
Hipótese. Na entrada já ordenada, o pivô fixo ($A[r]$) faz $n(n-1)/2$ comparações. Com pivô sorteado, a esperança é $2(n+1)H_n - 4n$ para **qualquer** entrada, inclusive esta. E as execuções devem se concentrar perto da esperança.

Método

1. Mesmo vetor ordenado $0, 1, \ldots, n-1$
2. n de 1.000 a 16.000
3. Histograma: 20.000 execuções, $n = 1000$, classes de 250
4. Semente fixa (`srand(12)`): reproduzível

EXPERIMENTO

Pivô fixo contra pivô sorteado (entrada ordenada)



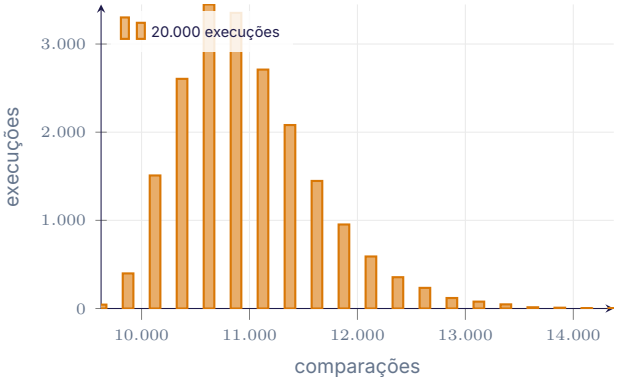
n	medido	medido $\div\, 2(n+1)H_n$
1.000	10.960	0
4.000	54.288	0
7.000	106.516	
10.000	156.841	
13.000	208.975	0
16.000	263.305	0

✓ desvio máx. 1,4%

Eixo y logarítmico: para $n = 16.000$, o pivô sorteado faz cerca de 500 vezes menos comparações.

EXPERIMENTO

Como as execuções se distribuem ($n = 1000$)



Média perto de
$2 \cdot 1001 \cdot H_{1000} - 4000 \approx 10.986$.
Nenhuma das 20.000 execuções
chegou perto do pior caso, 499.500.

EXPERIMENTO

O que o experimento mostrou

Na entrada que destrói o pivô fixo ($n(n-1)/2$ comparações), o pivô sorteado volta à esperança $2(n+1)H_n - 4n$. O histograma mostra execuções concentradas: a probabilidade de algo perto do pior caso é desprezível. **Aleatoriedade troca uma entrada ruim por uma sorte ruim, e sorte ruim é rara.**

Reproduza: `gcc -O2 aula12_experimento.c -o exp -lm && ./exp`



Conexões

- Segurança
- Redes
- Aprendizado de máquina
- Sistemas distribuídos

CONEXÕES

Onde isso aparece na computação

Segurança

Tabelas hash de linguagens (Python, Rust) sorteiam a semente da função hash a cada execução pelo mesmo motivo: impedir que um atacante escolha a entrada do pior caso.

Redes

O Ethernet usa recuo exponencial aleatório após colisões; balanceadores de carga usam "a melhor de duas escolhas aleatórias para distribuir requisições.

Aprendizado de máquina

Gradiente estocástico, *random forests* e *dropout*: a aleatoriedade torna algoritmos mais rápidos e mais robustos, com garantias em esperança.

Sistemas distribuídos

Protocolos de fofoca (*gossip*) e eleição de líder no Raft usam tempos aleatórios para evitar empates e espalhar informação em $O(\lg n)$ rodadas.

PARA CASA

Para casa

Exercícios

1. (7.3-1) Por que analisamos o tempo *esperado* e não o de pior caso do quicksort aleatorizado?
2. (7.3-2) Quantas chamadas a RANDOM ocorrem no pior e no melhor caso?
3. (7.4-2) Mostre que o melhor caso do quicksort é $\Omega(n \lg n)$.
4. (Problema 7-5) Implemente a partição pela mediana de 3 e meça a diferença.
5. No experimento, troque a semente e verifique que a média muda menos de 1%.

Leitura
Cormen et al., seções 7.3 e 7.4; apêndice C.3 (variáveis indicadoras).

Código
josedihego.net/estruturas